

Lecture Unit 4.1

MPI basics

Introduction

- This unit uses a MPI Hello World program to illustrate the basics of MPI
- We will see how to:
 - Initialise and finalise the MPI environment
 - Find the process rank and the total number of MPI processes
 - Compile the program
 - Run the program

Initialising and finalising the environment

- An MPI program needs to call two functions to initialise and finalise the MPI environment:
 - `MPI_Init`: call once to set up the MPI environment
 - `MPI_Finalize`: call once to cleanly shut down the MPI environment
- Call `MPI_Init` before any calling any other MPI functions
- Call `MPI_Finalize` after all the other MPI calls have been made

MPI_Init

- `MPI_Init`: Initialises the MPI execution environment
- Passes command line arguments to all MPI processes
- `int MPI_Init(int *argc, char ***argv)`
 - `argc`: pointer to the number of arguments
 - `argv`: pointer to the argument array
 - the return value of an MPI call is an integer error code

MPI_Finalize

- `MPI_Finalize`: Closes down the MPI execution environment
- `int MPI_Init(int *argc, char ***argv)`
 - takes no arguments
 - the return value of an MPI call is an integer error code

Size and rank

- We can call MPI functions to find out how many MPI processes there are and the rank of each process
- `MPI_Comm_size`: how many processes are there?
- `MPI_Comm_rank`: what is the rank of the calling process?
- Each MPI process has a rank between 0 and size-1

Communicators

- `MPI_Comm_rank` returns the rank of the calling process within the group of processes defined by a **communicator**
- The pre-defined communicator `MPI_COMM_WORLD` refers to all the concurrent processes which make up an MPI program
- We can define our own communicators if necessary and a process can be associated with more than one communicator

MPI_Comm_size

- `MPI_Comm_size`: reports the number of MPI processes in the specified communicator
- `int MPI_Comm_size(MPI_Comm comm, int*size)`
 - `comm`: communicator (e.g. `MPI_COMM_WORLD`)
 - `size`: number of processes in the communicator

MPI_Comm_rank

- `MPI_Comm_rank`: reports the rank of the calling process in the specified communicator
- `int MPI_Comm_rank(MPI_Comm comm, int*rank)`
 - `comm`: communicator (e.g. `MPI_COMM_WORLD`)
 - `rank`: rank of the calling processes in the communicator

Hello World program

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]){

    int myRank, nProcs;

    /* Initialise MPI environment*/
    MPI_Init(&argc, &argv);
    /* Find the rank of this process*/
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
    /* Find the total number of processes*/
    MPI_Comm_size(MPI_COMM_WORLD, &nProcs);
    /* Report rank and total number of processes*/
    printf("Hello World from process %d of %d\n", myRank, nProcs);
    /* Shut down the MPI environment*/
    MPI_Finalize();

    return 0;
}
```

MPI Header file

Number of command line arguments

Array containing arguments

Make command line arguments available to each process

MPI_COMM_WORLD refers to all MPI processes

Building and running

- With MPI we normally use a different compile command: e.g. `mpicc` for C
- This is a wrapper which calls a back end compiler and handles include paths, library paths, additional flags etc.
- To run an MPI process we use a command (e.g. `mpirun`) which starts multiple instances of the executable
- The number of processes to start is provided as a flag to `mpirun`
- `mpicc` and `mpirun` are commonly used names but this is not required by the standard. You may see different compile/run commands.

Demo

```
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ cat hello_mpi.c
/*****
Description: The MPI Hello World program from ECM3446/ECMM461 unit 4.1

Purpose: demonstrate the use of MPI by printing a Hello World message
        from each MPI process. The process also reports its rank and
        the total number of MPI processes.

Notes: compile with: mpicc -o hello_mpi hello_mpi.c
        run with:    mpirun -np 4 ./hello_mpi

Contact: D. Acreman, Exeter University
*****/

#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]){

    int myRank, nProcs;

    /* Initialise MPI environment*/
    MPI_Init(&argc, &argv);
    /* Find the rank of this process*/
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
    /* Find the total number of processes*/
    MPI_Comm_size(MPI_COMM_WORLD, &nProcs);
    /* Report rank and total number of processes*/
    printf("Hello World from process %d of %d\n", myRank, nProcs);
    /* Shut down the MPI environment*/
    MPI_Finalize();

    return 0;
}
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$
```

Demo

```
(OSS 2407.0) [jp1143@login02(exeter) ~]$ ls
HPC TeachingW2 TeachingW3 TeachingW4 Testing Workshops
(OSS 2407.0) [jp1143@login02(exeter) ~]$ cd TeachingW4
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ ls
bcast.c gather.c gather_malloc.c hello_mpi.c point-to-point.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpicc -o hello_mpi hello_mpi.c
-bash: mpicc: command not found
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ module load OpenMPI/4.1.5-GCC-12.3.0
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpicc -o hello_mpi hello_mpi.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ ls
bcast.c gather.c gather_malloc.c hello_mpi hello_mpi.c point-to-point.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$
```

Demo

```
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ module load OpenMPI/4.1.5-GCC-12.3.0
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpicc -o hello_mpi hello_mpi.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 4 hello_mpi
Hello World from process 3 of 4
Hello World from process 0 of 4
Hello World from process 1 of 4
Hello World from process 2 of 4
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ |
```

Demo

```
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ module load OpenMPI/4.1.5-GCC-12.3.0
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpicc -o hello_mpi hello_mpi.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 4 hello_mpi
Hello World from process 3 of 4
Hello World from process 0 of 4
Hello World from process 1 of 4
Hello World from process 2 of 4
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 8 hello_mpi
Hello World from process 7 of 8
Hello World from process 3 of 8
Hello World from process 4 of 8
Hello World from process 0 of 8
Hello World from process 1 of 8
Hello World from process 2 of 8
Hello World from process 5 of 8
Hello World from process 6 of 8
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ |
```

Unit Summary

- MPI housekeeping functions:
 - `MPI_Init`
 - `MPI_Finalize`
 - `MPI_Comm_size`
 - `MPI_Comm_rank`
- Compiling: `mpicc -o hello_mpi hello_mpi.c`
- Running: `mpirun -np 4 hello_mpi`